

Tecnólogo de Análisis y Desarrollo de Software

Ficha 2977389



Arquitectura y Documento Técnico del Sistema.

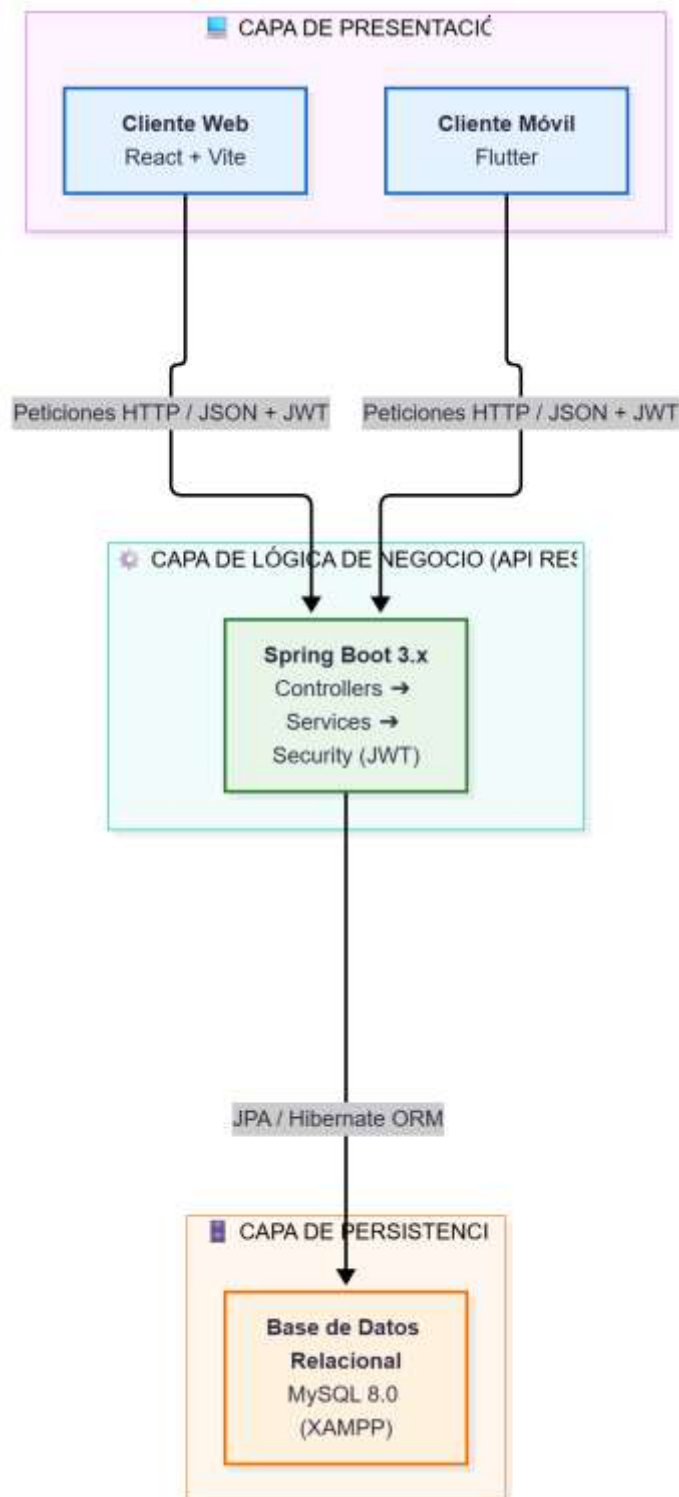
API REST Tienda de Lavadoras

Juan José Barrera Gómez

Servicio Nacional de Aprendizaje SENA

1. Visión General y Estilo Arquitectónico

El sistema transaccional "Tienda de Lavadoras" está diseñado bajo una **arquitectura desacoplada de tres capas** basada en el patrón de diseño **API REST (Representational State Transfer)**. Esta estructura garantiza la separación de responsabilidades, alta mantenibilidad, escalabilidad e interoperabilidad entre el servidor central y múltiples clientes (Web React y Móvil Flutter).



- **Desacoplamiento:** La lógica de negocio no depende de la interfaz gráfica; responde mediante intercambio de datos estandarizados en formato JSON.
- **Autenticación Stateless:** El servidor no guarda sesiones de usuario en memoria; cada petición valida la identidad mediante un token **JWT (JSON Web Token)** enviado en el encabezado HTTP `Authorization: Bearer <token>`.

2. Stack Tecnológico del Sistema

Capa / Módulo	Tecnología / Herramienta	Versión	Función Principal
Backend API	Java Development Kit (JDK)	17 LTS	Entorno de ejecución principal de la API.
Framework Backend	Spring Boot	3.x	Framework para construir microservicios y REST APIs.
Seguridad	Spring Security + JWT	6.x	Control de acceso por roles (ROLE_ADMIN, ROLE_CLIENTE).
Persistencia ORM	Spring Data JPA / Hibernate	3.x	Mapeo objeto-relacional y abstracción de consultas SQL.
Base de Datos	MySQL Server (XAMPP)	8.0	Motor relacional para almacenamiento del catálogo y transacciones.
Cliente Web	React + Vite + TailwindCSS	18.x	Panel web de administración y catálogo de productos para clientes.
Cliente Móvil	Flutter SDK	3.x	App móvil multiplataforma para consulta y gestión de pedidos.
Documentación API	OpenAPI 3 / Swagger UI	2.x	Interfaz interactiva de prueba y documentación de endpoints.

3. Modelo Relacional de Base de Datos

La base de datos relacional utiliza el motor **InnoDB** en MySQL para garantizar integridad referencial y soporte a transacciones (propiedades ACID).

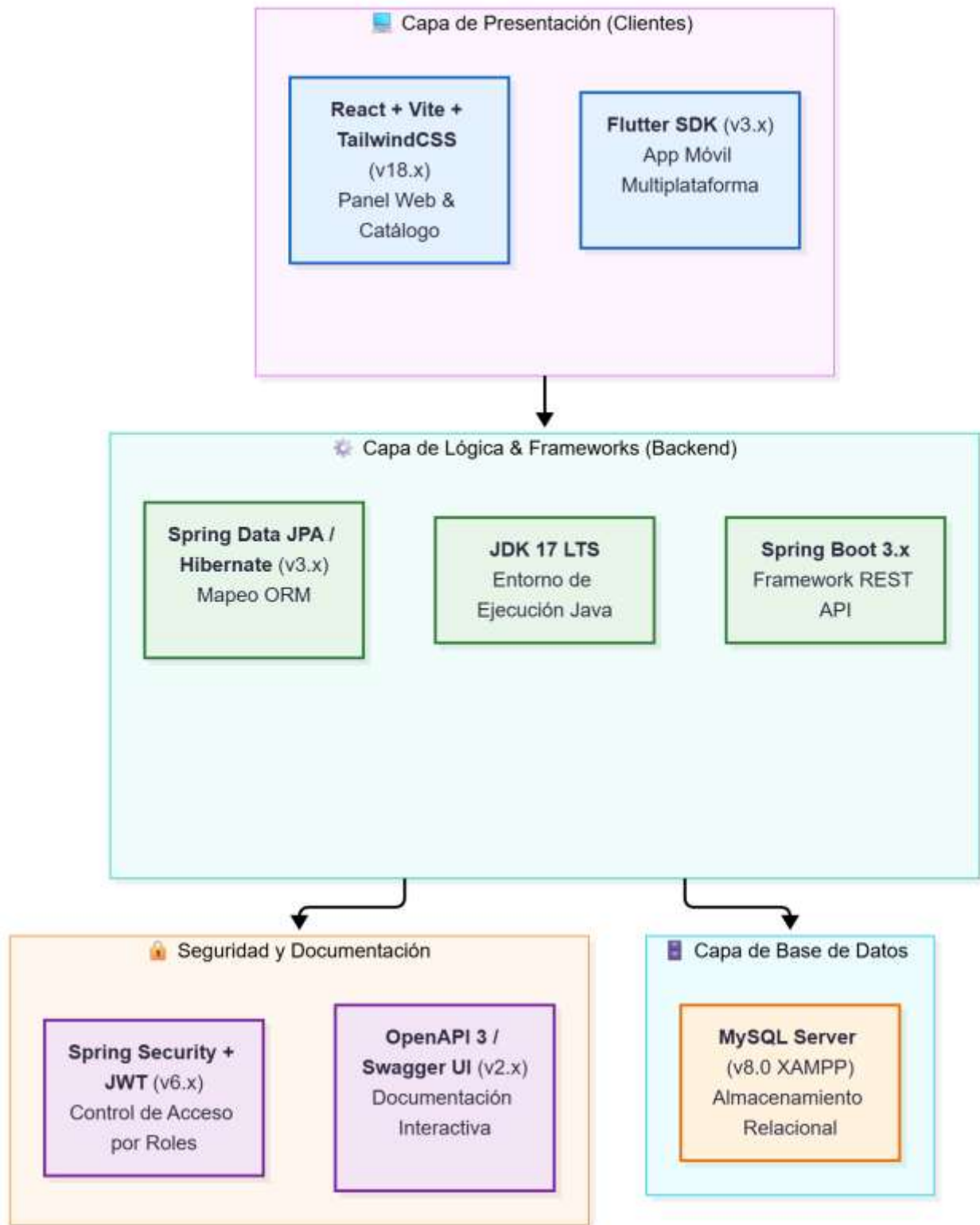
3.1 Estructura de Tablas Principales

1. **usuarios:** Almacena clientes y administradores con contraseñas encriptadas mediante **BCrypt**.
 - o `id` (PK, BigInt, Auto-increment)
 - o `nombre` (VarChar 100, Not Null)
 - o `email` (VarChar 100, Unique, Not Null)
 - o `password` (VarChar 255, Not Null)
 - o `rol` (Enum: 'ROLE_ADMIN', 'ROLE_CLIENTE', Not Null)
2. **lavadoras:** Mantiene el catálogo de productos disponibles en el inventario.
 - o `id` (PK, BigInt, Auto-increment)
 - o `marca` (VarChar 50, Not Null)
 - o `modelo` (VarChar 50, Not Null)
 - o `capacidad_kg` (Decimal 5,2, Not Null)
 - o `precio` (Decimal 10,2, Not Null)
 - o `stock` (Int, Not Null)
 - o `imagen_url` (VarChar 255)
3. **pedidos:** Contiene el encabezado de la transacción de compra efectuada por un usuario.
 - o `id` (PK, BigInt, Auto-increment)
 - o `usuario_id` (FK → usuarios.id, Not Null)
 - o `fecha_creacion` (DateTime, Not Null)
 - o `total` (Decimal 10,2, Not Null)
 - o `estado` (Enum: 'PENDIENTE', 'ENVIADO', 'CANCELADO', Not Null)
4. **detalles_pedido:** Almacena las líneas de producto asociadas a cada orden de compra.
 - o `id` (PK, BigInt, Auto-increment)
 - o `pedido_id` (FK → pedidos.id, Not Null)
 - o `lavadora_id` (FK → lavadoras.id, Not Null)
 - o `cantidad` (Int, Not Null)
 - o `precio_unitario` (Decimal 10,2, Not Null)

4. Especificación de Endpoints de la API REST

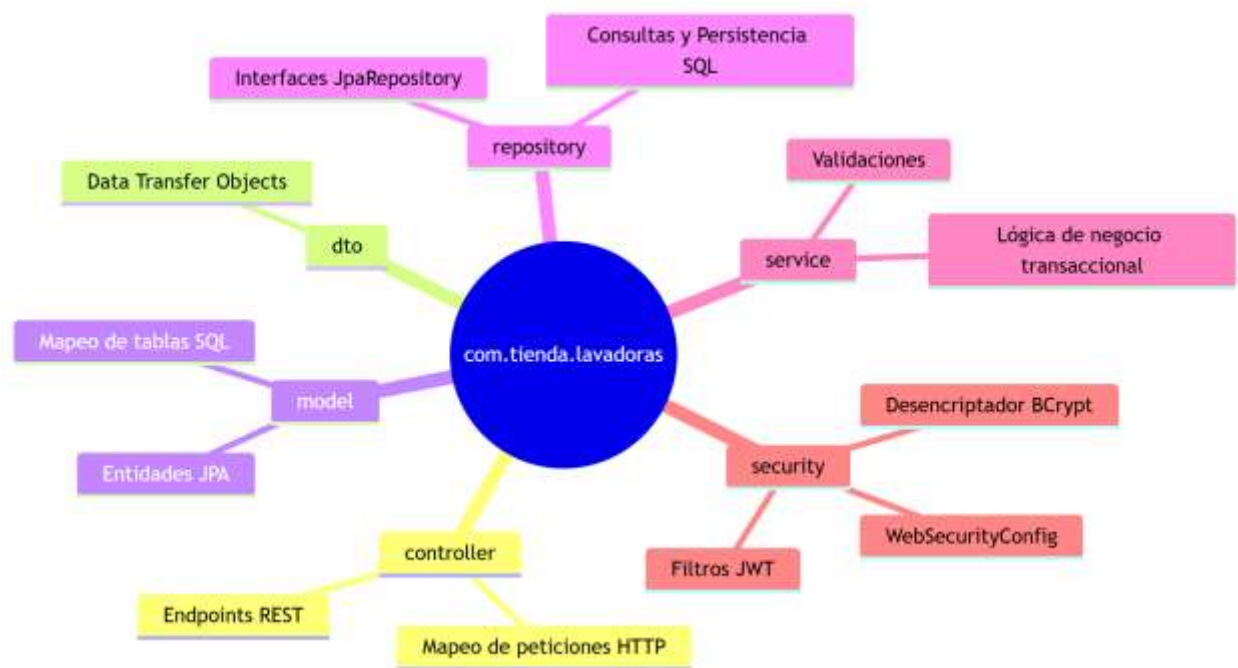
Método	Endpoint	Permiso / Rol	Descripción de la Operación
POST	/api/auth/register	Público	Registro de nuevos clientes en el sistema.
POST	/api/auth/login	Público	Autenticación de usuarios; devuelve el token JWT.
GET	/api/lavadoras	Público	Obtención del catálogo general de lavadoras.
POST	/api/lavadoras	ROLE_ADMIN	Registro de una nueva lavadora en

			inventario.
PUT	/api/lavadoras/{id}	ROLE_ADMIN	Actualización de precio, características o stock.
DELETE	/api/lavadoras/{id}	ROLE_ADMIN	Eliminación lógica/física de un producto.
POST	/api/pedidos	ROLE_CLIENTE	Generación de una nueva orden de compra con descuento de stock.
GET	/api/pedidos/mis-pedidos	ROLE_CLIENTE	Consulta de historial de compras del usuario autenticado.
PUT	/api/pedidos/{id}/estado	ROLE_ADMIN	Cambio de estado de una orden (ENVIADO, CANCELADO).



5. Patrones de Diseño y Estructura del Código Source (Backend)

La API organizada en tres capas sigue el estándar de paquetes de Spring Boot:



- **Patrón DTO (Data Transfer Object):** Evita exponer directamente las entidades JPA en las peticiones web, protegiendo campos sensibles como hashes de contraseñas.
- **Patrón Repository:** Utiliza Spring Data JPA para realizar operaciones CRUD sin escribir consultas SQL manuales propensas a inyecciones.
- **Inyección de Dependencias (DI):** Administrada por el contenedor de Inversión de Control (IoC) de Spring mediante la anotación `@Autowired`.

Referencias

Bloch, J. (2018). *Effective Java (3rd ed.)*. Addison-Wesley Professional.

Walls, C. (2022). *Spring in action (6th ed.)*. Manning Publications.